

BANCO DE DADOS

Transações



Júlio César Chaves

Julho, 2026

julio.cesar.chaves@prof.fgv.edu.br

Mecanismos transacionais:

ACID

Atomicidade

Depois que inicia-se uma transação, ou ela finaliza com sucesso, ou ela se desfaz completamente.

Consistência

Conceito em que o SGBD sai de um estado válido apenas para ir para outro estado válido.

Além dos mecanismos internos, o modelo do banco de dados precisa refletir a visão e as regras de negócio.

Mecanismos transacionais:

ACID

Isolamento

Conceito em que cada transação ocorre de forma isolada e independente de outras transações, que podem estar ocorrendo ao mesmo tempo e nos mesmos objetos.

Na prática, o isolamento é obtido através de bloqueios impeditivos.

Durabilidade

Conceito referente à durabilidade da transação, mesmo em situações de desastre.

Um SGBD possui áreas específicas para o registro de todas as transações, que podem ser reaplicadas durante uma restauração ou aplicadas em um banco de contingência.

ACID na prática

Imagine um sistema bancário processando uma transferência de fundos:

Atomicidade: toda a transferência é concluída ou falha totalmente

Consistência: os saldos das contas permanecem válidos durante toda a transação

Isolamento: as transferências simultâneas não interferem umas nas outras

Durabilidade: Uma vez confirmada, a transferência é permanente e sobrevive a falhas do sistema

Levels of Consistency in SQL-92

▪ *Do menos ao mais restritivo*

- **Read uncommitted** — registros não confirmados (committed) podem ser lidos.
- **Read committed** — apenas registros confirmados (committed) podem ser lidos, é o nível de isolamento padrão para o SQL Server. Impede a realização de leituras sujas especificando que as instruções **não** podem ler valores de dados que foram modificados, mas ainda não confirmados por outras transações. Outras transações ainda podem modificar, inserir ou excluir dados entre execuções de instruções individuais dentro da transação atual, resultando em leituras não repetíveis ou dados fantasmas. Simplesmente restringe o leitor de fazer qualquer leitura intermediária, não comprometida e 'suja'. Não faz nenhuma promessa de que, se a transação reemitir a leitura, encontrará os mesmos dados, os dados estarão livres para serem alterados depois de lidos.
- **Repeatable read** — além de ler apenas registros confirmados (committed), especifica que nenhuma outra transação pode modificar nem excluir dados que tenham sido lidos pela transação atual, até que a transação atual seja confirmada. Os bloqueios compartilhados nos dados lidos são mantidos até o término da transação, em vez de serem liberados ao final de cada instrução. No entanto, uma transação pode não ser serializável – ela pode encontrar alguns registros inseridos por uma transação, mas não encontrar outros.
- **Serializable** — bloqueia intervalos de chaves inteiros até que a transação seja concluída. Ele engloba REPEATABLE READ e adiciona a restrição de que outras transações não podem inserir novas linhas em intervalos que foram lidos pela transação até que a transação esteja concluída.

Níveis de Isolamento (SQL-92) na prática

Cenário base:

Conta A: saldo R\$ 1.000

Conta B: saldo R\$ 500

Transação T1: transfere R\$ 200 de A para B (debita A, credita B, depois COMMIT)

Transação T2: uma transação de consulta/relatório, rodando ao mesmo tempo que T1

A ideia é mostrar o que T2 enxerga em cada nível, conforme T1 vai executando.

1. Read Uncommitted (menos restritivo)

T1 já debitou R\$200 de A (saldo agora R\$800 na memória/transação), mas ainda não deu COMMIT.

T2 lê o saldo de A e vê R\$ 800 — um valor que ainda não é oficial.

Se T1 sofrer um ROLLBACK (por exemplo, a conta B não existir), o saldo de A volta a R\$1.000.

Resultado: T2 baseou uma decisão (ex: gerar um relatório, negar um crédito) em um dado que nunca existiu oficialmente.

Isso é a leitura suja (dirty read). É como abrir a porta de um cofre no meio de uma operação e ver dinheiro sendo contado, sem saber se a operação vai ser confirmada ou desfeita.

2. Read Committed (padrão do SQL Server)

Agora T2 não pode mais ver o valor R\$800 antes do commit de T1. Ela só enxerga R\$1.000 até que T1 confirme.

Mas repare no problema seguinte, dentro da mesma transação T2:

T2 lê o saldo de A → R\$ 1.000 (primeira leitura, T1 ainda não comitou)

T1 completa a transferência e dá COMMIT (A = R\$800)

T2, na mesma transação, lê o saldo de A novamente → agora vê R\$ 800

T2 obteve dois valores diferentes para o mesmo dado, na mesma transação, sem nunca ter dado commit. Isso é a leitura não repetível (non-repeatable read). O nível impede a leitura suja, mas não impede que o dado mude "debaixo do seu pé" entre uma leitura e outra.

3. Repeatable Read

Agora, quando T2 lê o saldo de A pela primeira vez, um bloqueio compartilhado é colocado sobre esse registro. T1 não consegue alterar/debitar a conta A enquanto T2 não terminar.

T2 lê A → R\$1.000

T1 tenta debitar A → fica bloqueada, esperando T2 finalizar

T2 lê A de novo → ainda R\$ 1.000 (valor garantidamente repetível)

T2 finaliza → T1 é liberada e prossegue

Isso resolve a leitura não repetível. Mas ainda existe uma brecha: imagine que T2, em vez de ler uma linha específica, execute:

SELECT COUNT(*) FROM Contas WHERE saldo > 500;

Primeira execução: retorna 3 contas

Enquanto isso, T1 insere uma nova conta (Conta D, saldo R\$600) e dá commit — essa é uma linha nova, não uma que T2 já tinha lido, então o bloqueio de Repeatable Read não a impede

T2 executa a mesma consulta de novo: agora retorna 4 contas

Esse fantasma (phantom read) surge porque Repeatable Read tranca linhas já lidas, mas não tranca o intervalo de valores da consulta (saldo > 500), permitindo que novas linhas "apareçam" dentro dele.

4. Serializable (mais restritivo)

Aqui o SGBD trava o intervalo de chaves inteiro correspondente à condição da consulta (saldo > 500), não apenas as linhas já existentes.

T2 executa `SELECT COUNT(*) FROM Contas WHERE saldo > 500` → retorna 3 contas

T1 tenta inserir a Conta D com saldo R\$600 → como esse valor cai dentro do intervalo bloqueado por T2, a inserção fica bloqueada até T2 terminar

T2 repete a consulta → continua retornando 3 contas, garantido

Só depois que T2 finaliza, T1 é liberada para inserir a Conta D

Do ponto de vista lógico, é como se as transações concorrentes tivessem rodado uma de cada vez, em sequência (daí o nome serializable) — mesmo que fisicamente tenham sido processadas em paralelo.

Trade-off: quanto mais alto o nível, mais **consistência garantida**, mas também mais **bloqueios** e menos **concorrência** — transações demoram mais e podem até entrar em espera umas das outras. É o clássico equilíbrio entre **consistência** e **desempenho/concorrência** em bancos de dados relacionais.

Visão de consistência no SQL Server

--Abra uma sessão para ler a tabela de apartamentos com um temporizador de 1min

`SET SEARCH_PATH = HAFH;`

-- Iniciar uma transação

`BEGIN;`

-- Selecionar dados da tabela APARTMENT

`SELECT * FROM APARTMENT;`

-- aguarde para dar o rollback

`ROLLBACK;`

Use o banco do HAFH. Vamos ler os apartamentos em uma sessão, além de fazer algumas modificações em outra sessão.

Primeiramente confira o estado atual da tabela APARTMENT.

	AptNo	ANoOfBedrooms	BuildingID	CCID
1	11	2	B2	C222
2	11	2	B3	C777
3	11	2	B4	C777
4	21	1	B1	C111
5	31	2	B2	NULL
6	41	1	B1	NULL

Visão de consistência no SQL Server

Results		Messages		
	AptNo	ANoOfBedrooms	BuildingID	CCID
1	11	2	B2	C222
2	11	2	B3	C777
3	11	2	B4	C777
4	21	1	B1	C111
5	31	2	B2	C111
6	41	1	B1	NULL
7	51	2	B1	C222

--Abra outra sessão para fazermos modificações na tabela de apartamentos enquanto a outra sessão está com transação aberta

SET *SEARCH_PATH* = *HAFH*;

-- Inserir dados na tabela APARTMENT

BEGIN;

INSERT INTO APARTMENT

(AptNo,
ANoOfBedrooms,
BuildingID,
CCID)

VALUES

('51',
2,
'B1',
'C222');

-- VOLTE NA SESSÃO ANTERIOR E FAÇA SELECT

-- Atualizar um registro na tabela APARTMENT

UPDATE APARTMENT SET CCID='C111'

WHERE AptNo='31' AND BuildingID='B2';

-- Selecionar todos os registros da tabela APARTMENT

SELECT * FROM APARTMENT;

Visão de consistência no SQL Server

Voltamos à sessão anterior depois de um minuto e vemos o resultado do SELECT anterior as modificações.

Results		Messages		
	AptNo	ANoOfBedrooms	BuildingID	CCID
1	11	2	B2	C222
2	11	2	B3	C777
3	11	2	B4	C777
4	21	1	B1	C111
5	31	2	B2	NULL
6	41	1	B1	NULL

Bloqueio impeditivo

--Abra uma sessão para bloquear a tabela de apartamentos com um temporizador de 1min

BEGIN;

SELECT * FROM APARTMENT FOR UPDATE;

SELECT pg_sleep(60);

ROLLBACK;

O SGBD oferece a opção de "forçar" o bloqueio de uma tabela, assegurando que nenhuma outra sessão poderá ler ou modificar dados enquanto a sua sessão estiver com o bloqueio transacional ativado.

--Abra outra sessão para tente atualizar um registro

UPDATE APARTMENT SET CCID='C111'

WHERE AptNo='31' AND BuildingID='B2';

Bloqueio impeditivo (update)

O que acontece?

--Abra uma sessão para atualizar um registro na tabela de apartamentos com um temporizador de 1min

BEGIN;

UPDATE APARTMENT SET CCID='C222'

WHERE AptNo='31' AND BuildingID='B2';

SELECT pg_sleep(60);

COMMIT;

--Abra outra sessão para tentar ler a tabela de apartamentos

SELECT * FROM APARTMENT;

--tente atualizar

UPDATE APARTMENT SET CCID='C111'

WHERE AptNo='31' AND BuildingID='B2';

--tente inserir

INSERT INTO APARTMENT

(AptNo

,ANoOfBedrooms

,BuildingID

,CCID)

VALUES ('61',2,'B1','C222');

Bloqueio impeditivo (leitura “suja”)

--Abra uma sessão para atualizar um registro na tabela de apartamentos com um temporizador de 1min

BEGIN TRAN

UPDATE [dbo].[APARTMENT] SET [CCID]='C222'

WHERE [AptNo]='31' AND [BuildingID]='B2'

GO

WAITFOR DELAY '00:01:00'

COMMIT TRAN

GO

Não suportado no Postgresql.

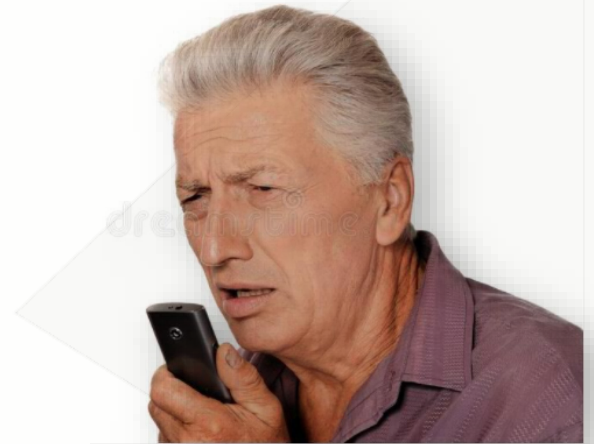
--Abra outra sessão para tentar ler a tabela de apartamentos indicando que autoriza a leitura “suja”

SELECT * FROM APARTMENT WITH (NOLOCK)

GO

Mecanismos transacionais:

Deadlock



Bloqueio insolúvel

Abra uma sessão e inicie uma transação sem comando para finalizar

```
BEGIN;  
UPDATE APARTMENT SET  
CCID='C111'  
WHERE AptNo='31' AND  
BuildingID='B2';
```

Abra outra sessão e inicie a mesma transação sem comando para finalizar

```
BEGIN;  
UPDATE APARTMENT SET  
CCID='C111'  
WHERE AptNo='31' AND  
BuildingID='B2';
```

Tipos de processamento de BD: BASE

Você sabia?

O termo BASE foi veio de Eric Brewer como parte do teorema CAP.

Os bancos de dados NoSQL geralmente implementam princípios BASE.

Os sistemas BASE geralmente podem alcançar maior desempenho e escalabilidade do que os sistemas ACID.

Alguns sistemas implementam um espectro entre ACID e BASE, em vez de estritamente um ou outro.

Os princípios BASE são fundamentais para muitos serviços baseados em nuvem e sistemas distribuídos.

O BASE (Basically Available, Soft State, Eventual Consistency) surgiu como um contraponto ao ACID, abordando os desafios de grandes volumes de dados, dados não estruturados e a necessidade de escalabilidade em ambientes de dados modernos.

Princípios fundamentais:

Basicamente disponível: garante algum nível de acesso aos dados, mesmo durante falhas de alguns nós

Estado flexível: reconhece o estado constante de fluxo dos dados

Consistência eventual: garante a consistência dos dados entre os nós ao longo do tempo, não imediatamente

Os sistemas BASE são predominantes em cenários de Big Data, especialmente para grandes organizações online e empresas de mídia social, onde a precisão em tempo real nem sempre é crítica.

Exemplo fictício

Imagine uma plataforma de mídia social popular:

Basicamente disponível: os usuários podem postar atualizações mesmo que alguns servidores estejam inativos

Estado flexível: as contagens de curtidas podem diferir temporariamente entre os dispositivos do usuário

Consistência eventual: todos os seguidores verão a postagem, mas não necessariamente simultaneamente

Tipos de processamento de BD: CAP

O Teorema CAP, também conhecido como Teorema de Brewer, aborda os desafios inerentes aos sistemas distribuídos:

Consistência: o sistema se comporta conforme o esperado em todos os momentos

Disponibilidade: O sistema responde a todas as solicitações

Tolerância de partição: O sistema continua a operação durante falhas parciais

O teorema postula que um sistema distribuído pode atingir no máximo duas dessas três propriedades simultaneamente, levando ao paradigma "escolha dois".

Exemplo fictício

Considere uma plataforma global de comércio eletrônico:

Consistência + Disponibilidade: inventário preciso, mas pode falhar durante indisponibilidades parciais devido a problemas de comunicação

Disponibilidade + Tolerância de Partição: Sempre operacional, mas pode mostrar inventário desatualizado

Consistência + Tolerância de Partição: Dados precisos, mas podem ficar temporariamente indisponíveis

O teorema CAP

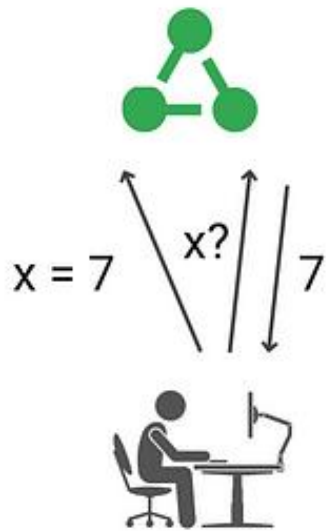
*O Teorema CAP, proposto por Eric Brewer, é um princípio fundamental no design de sistemas distribuídos, estabelecendo que é impossível garantir simultaneamente **Consistência**, **Disponibilidade** e Tolerância via Partições.*

Origem: Formalizado por Eric Brewer (2000) e comprovado por Gilbert e Lynch (2002).

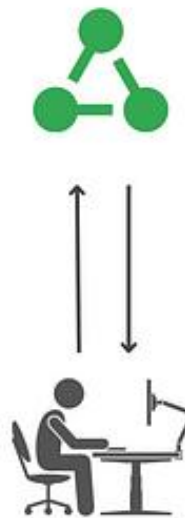
Premissa: Sistemas distribuídos só podem garantir dois dos três pilares simultaneamente.

Analogia: "Bom, rápido e barato: escolha dois"

Consistency



Availability



Partition Tolerance



https://medium.com/@gurpreet.singh_89/understanding-the-cap-theorem-consistency-availability-and-partition-tolerance-e7faa5103638

<https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>

